Classes e Objetos

UA.DETI.POO

O que é uma classe?

- Classes são especificações para criar objetos
- Uma classe representa um tipo de dados complexo
- Classes descrevem
  - Tipos dos dados que a caracterizam e depois vão compor o objeto (i.e., o que podem armazenar)
  - Métodos que operam sobre (i) as características daquele objeto, (ii) características comuns a todos os objetos da classe, (iii) parâmetros de entrada dados na invocação do método (i.e., o que podem fazer)

Classes são o esqueleto para construir objetos

- Podem ter:
  - Valores *(i.e., as propriedades que a caracterizam)*
  - Métodos
- **Necessário "alocar" espaço e construir o objeto (i.e., new)**

Java

```
1  class Car {
2    String model;
3    int year;
4    Car(String m, int yr) {model=m; year=yr;}
5  }
6
7  public class CarTest {
8    public static void main(String[] args) {
9      Car toyota = new Car("Toyota Camry",2009);
10     Car ford = new Car("Ford F-150",2023);
11     String desc = "This code is about cars!";
12   }
13 }
```

Edit Code & Get AI Help

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (7 steps)

Frames

main:12

toyota	
ford	
desc	"This code is about cars!"

Objects

Car instance

model	"Toyota Camry"
year	2009

Car instance

model	"Ford F-150"
year	2023

https://pythontutor.com/articles/java-visualizer.html

O que é uma classe?

- Exemplo:

```
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    public int getAge() {
        int currentYear = java.time.Year.now().getValue();
        return getAge( currentYear );
    }

    public int getAge( int currentYear) {
        return currentYear - year;
    }
}
```

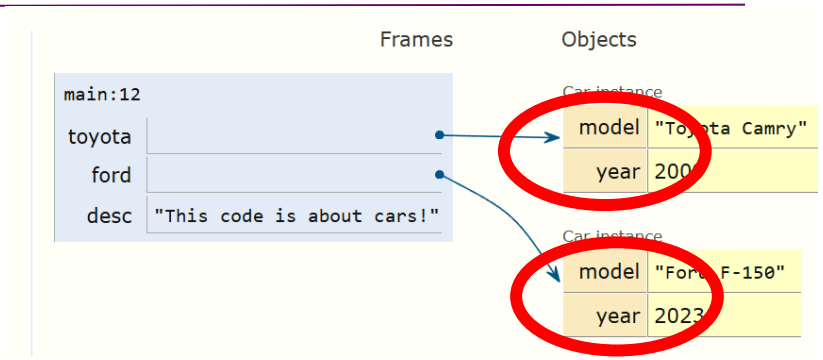
Classes são “blueprints” para objetos

```
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    public int getAge() {
        int currentYear = java.time.Year.now().getValue();
        return getAge( currentYear );
    }

    public int getAge( int currentYear) {
        return currentYear - year;
    }
}
```

O que é uma classe?

- Exemplo:

```
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    public int getAge() {
        int currentYear = java.time.Year.now().getValue();
        return getAge( currentYear );
    }

    public int getAge( int currentYear) {
        return currentYear - year;
    }
}
```



O que pode conter uma classe

- A definição de uma classe pode incluir:
  - zero ou mais declarações de atributos de dados
  - zero ou mais definições de métodos
  - zero ou mais construtores
  - zero ou mais blocos de inicialização static
  - zero ou mais definições de classes ou interfaces internas
- Esses elementos só podem ocorrer dentro do bloco ‘class NomeDaClasse { ... }’
  - "tudo pertence" a alguma classe
  - apenas ‘import’ e ‘package’ podem ocorrer fora de uma declaração ‘class’ (ou ‘interface’)

O que é uma classe?

- Exemplo:

```
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    public int getAge() {
        int currentYear = java.time.Year.now().getValue();
        return getAge( currentYear );
    }

    public int getAge( int currentYear) {
        return currentYear - year;
    }
}
```

O que é uma classe?

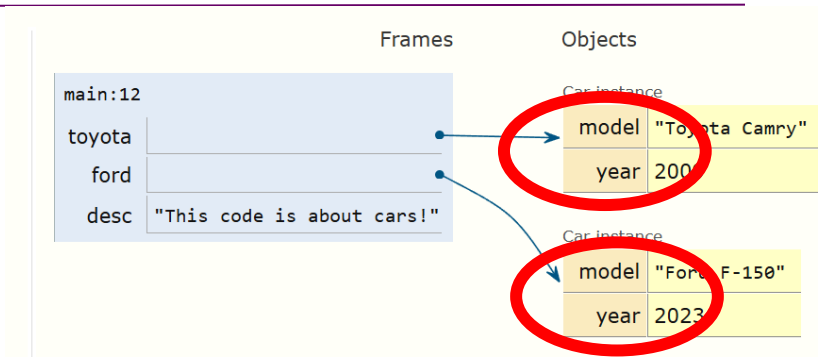
- Exemplo:

atributos

constructor

métodos

```
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    public int getAge() {
        int currentYear = java.time.Year.now().getValue();
        return getAge( currentYear );
    }

    public int getAge( int currentYear) {
        return currentYear - year;
    }
}
```



Propriedades / membros

Classes são “blueprints” para objetos

- Podem **ter valores / properties**

```
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    public int getAge() {
        int currentYear = java.time.Year.now().getValue();
        return getAge( currentYear );
    }

    public int getAge( int currentYear) {
        return currentYear - year;
    }
}
```

Valores de omissão para tipos primitivos

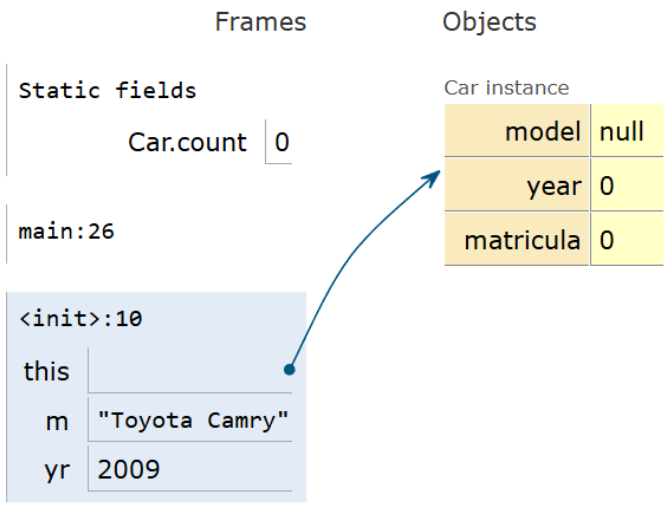
- Se uma variável for utilizada como membro de uma classe o compilador encarrega-se de inicializá-la por omissão
  - Isto não é garantido no caso de variáveis locais pelo que devemos sempre inicializar todas as variáveis

Tipo	Valor por omissão
boolean	false
char	'\u0000'
byte	(byte)0
short	(short)0
int	0
long	0L
float	0.0f
double	0.0
(outros tipos)	null

Classes podem ter membros estáticos

- manter informação da classe
  - Partilhar valores ao nível da classe (similar variável global)
  - e.g., constantes, contadores, contexto,

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

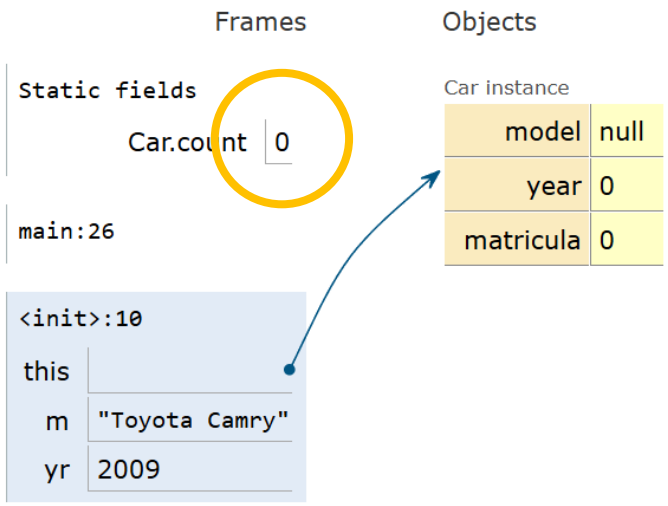



```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+" anos.");
    System.out.println( "O Ford é de "+ford.getAge()+" anos.");
    System.out.println("tenho "+Car.count+" carros");
  }
}
```

Classes podem membros estáticos

- manter informação da classe
  - Partilhar valores ao nível da classe (similar variável global)
  - e.g., constantes, contadores, contexto,

```
public class Car {
  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```




```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+" anos.");
    System.out.println( "O Ford é de "+ford.getAge()+" anos.");
    System.out.println("tenho "+Car.count+" carros");
  }
}
```

Classes podem membros estáticos

- manter informação da classe
  - Partilhar valores ao nível da classe (similar variável global)
  - e.g., constantes, contadores, contexto,

2 x

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int y) {model m, year y};
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+"
    System.out.println( "O Ford é de "+ford.getAge()+" anc
    System.out.println("tenho "+Car.count+" carros");
  }
}
```

Print output (drag lower right corner to resize)

```
O Toyota é de 16 anos.
O Ford é de 2 anos.
tenho 2 carros
```

Frames

Static fields

Car.count 2

main:32

toyota	
ford	
desc	"This code is about cars!"

Objects

Car instance

model	"Toyota Camry"
year	2009
matricula	0

Car instance

model	"Ford F-150"
year	2023
matricula	1

Classes podem membros estáticos

- manter informação da classe
  - Partilhar valores ao nível da classe (similar variável global)
  - e.g., constantes, contadores, contexto,

2 x

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+"
    System.out.println( "O For  .getAge()+" anc
    System.out.println("tenho  +Car.count+  carros");
  }
}
```

Print output (drag lower right corner to resize)

```
O Toyota é de 16 anos.
O Ford é de 2 anos.
tenho 2 carros
```

Frames

Static fields

Car.count 2

main:32

toyota

ford

desc "This code is about cars!"

Objects

Car instance

model	"Toyota Camry"
year	2009
matricula	0

Car instance

model	"Ford F-150"
year	2023
matricula	1

métodos

Métodos

- Métodos, mensagens, funções, procedimentos
- A invocação é sempre efetuada através da notação de ponto.

```
oneBook.setTitle("Turismo em Aveiro");
otherBook.setPubYear(2025);
```

- O recetor da mensagem está sempre à esquerda.
- O recetor é sempre uma classe ou uma referência para um objeto.

```
Math.sqrt(34);
otherBook.setPubYear(2025);
```

Exemplo de classe

```
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    public int getAge() {
        int currentYear = java.time.Year.now().getValue();
        return getAge( currentYear );
    }

    public int getAge( int currentYear) {
        return currentYear - year;
    }
}
```

```
public class Book {
   String title;
   int pubYear;

   String getTitle() {
      return title;
   }
   int getPubYear() {
      return pubYear;
   }
   void setTitle(String atitle) {
      title = atitle;
   }
   void setPubYear(int apubYear) {
      pubYear = apubYear;
   }
}
```

Objectos

Objetos

- Objetos são instâncias de classes

```
Book oneBook = new Book();
Book otherBook = new Book();
Book book3 = new Book();
```

- Todos os objetos são manipulados através de referências

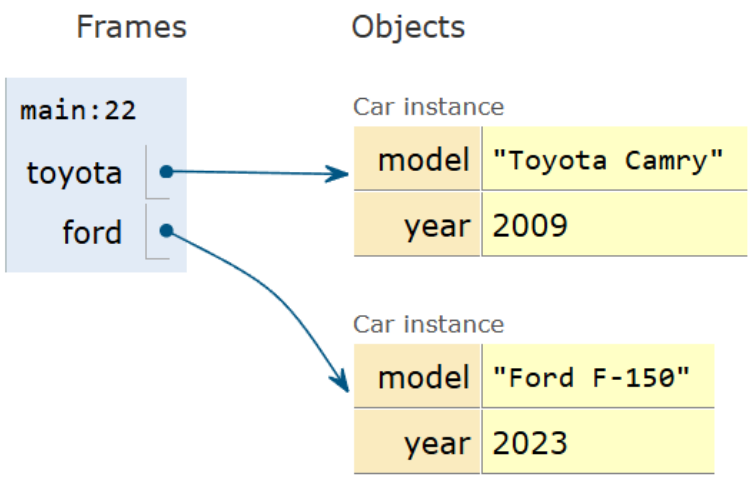
```
Pessoa nome1, nome2;
nome1 = new Pessoa("Manuel");
nome2 = nome1;
```

- Todos os objetos devem ser explicitamente criados.

```
Circulo c1 = new Circulo(p1, 5);
String s = "Livro";  // Strings são excepção!
```

Objetos são tipos de dados por referência

```
class Car {
    String model;
    int year;
    int getYear()  { return this.year;  }

    Car(String m, int yr) {model=m; year=yr;}

    int getAge() {
      int currentYear = java.util.Calendar.getInstance().get(java.util.Calendar.Y
      return getAge(currentYear);

    }
    int getAge(int currentYear) {
      return currentYear - year;
    }
  }
```




```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+" anos.");
    System.out.println( "O Ford é de "+ford.getAge()+" anos.");
  }
}
```

Objetos são tipos de dados por referência

- Variáveis não têm "os dados", são referências em memória para a localização "dos dados".

Java

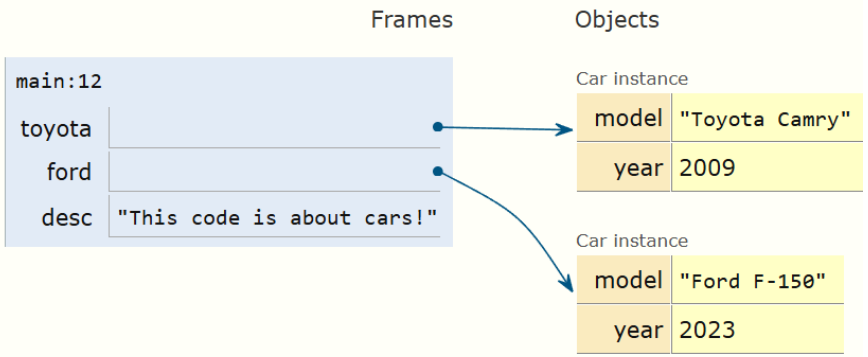
```
1  class Car {
2    String model;
3    int year;
4    Car(String m, int yr) {model=m; year=yr;}
5  }
6
7  public class CarTest {
8    public static void main(String[] args) {
9      Car toyota = new Car("Toyota Camry",2009);
10     Car ford = new Car("Ford F-150",2023);
11     String desc = "This code is about cars!";
12   }
13 }
```

Edit Code & Get AI Help

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (7 steps)



https://pythontutor.com/articles/java-visualizer.html

Objetos são tipos de dados por referência

- Variáveis não têm "os dados", são referências em memória para a localização "dos dados".

https://pythontutor.com/articles/java-visualizer.html

Objetos são tipos de dados por referência

- Variáveis não têm "os dados", são referências em memória para a localização "dos dados".

Java

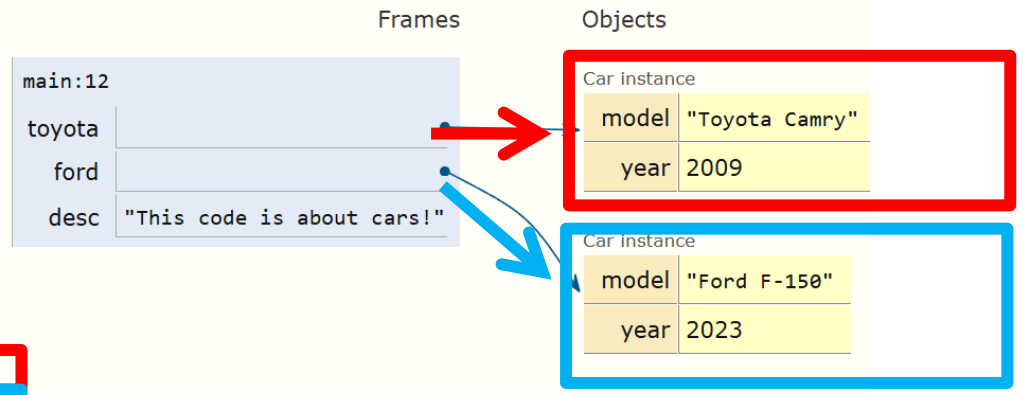
```
1  class Car {
2    String model;
3    int year;
4    Car(String m, int yr) {model=m; year=yr;}
5  }
6
7  public class CarTest {
8    public static void main(String[] args) {
9      Car toyota = new Car("Toyota Camry",2009);
10     Car ford = new Car("Ford F-150",2023);
11     String desc = "This code is about cars!";
12   }
13 }
```

Edit Code & Get AI Help

line that just executed
next line to execute

<< First | < Prev | Next > | Last >>

Done running (7 steps)



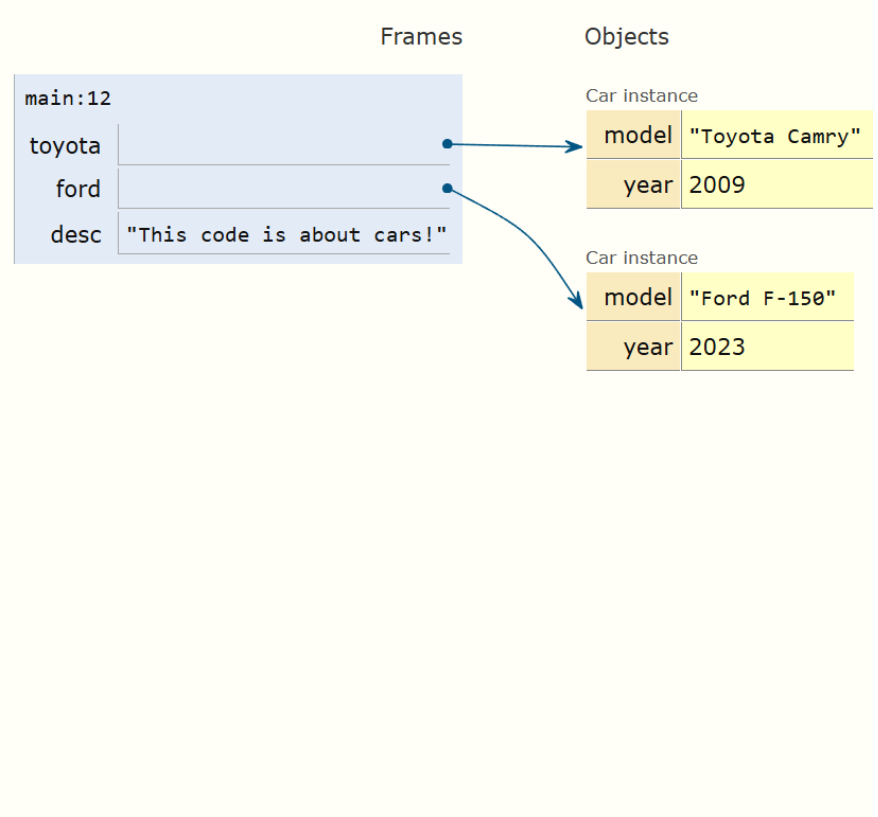
https://pythontutor.com/articles/java-visualizer.html

Objetos são tipos de dados por referência

- **Podem ter métodos**
  - **Sobre objetos**

```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+" anos.");
    System.out.println( "O Ford é de "+ford.getAge()+" anos.");
  }
}
```

```
class Car {
    String model;
    int year;
    int getYear()  { return this.year;  }

    Car(String m, int yr) {model=m; year=yr;}

    int getAge() {
      int currentYear = java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
      return getAge(currentYear);

    }
    int getAge(int currentYear) {
      return currentYear - year;
    }
}
```

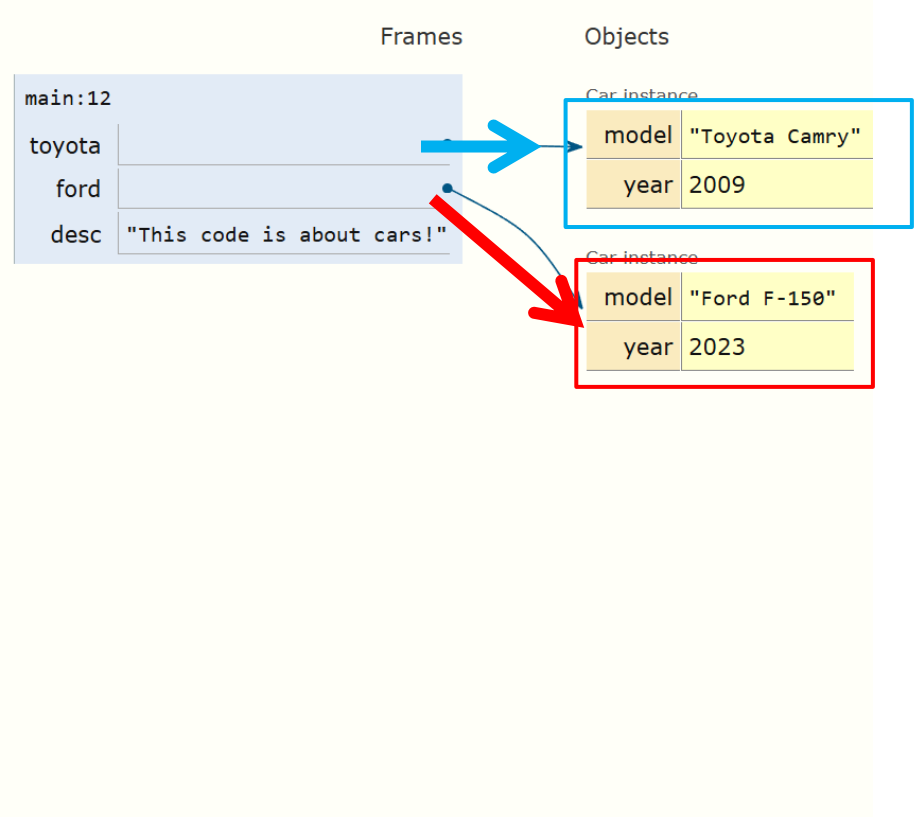


Objetos são tipos de dados por referência

- **Podem ter métodos**
  - **Sobre objetos**

```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+" anos.");
    System.out.println( "O Ford é de "+ford.getAge()+" anos.");
  }
}
```

```
class Car {
    String model;
    int year;
    int getYear()  { return this.year;  }

    Car(String m, int yr) {model=m; year=yr;}

    int getAge() {
      int currentYear = java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
      return getAge(currentYear);

    }
    int getAge(int currentYear) {
      return currentYear - year;
    }
}
```

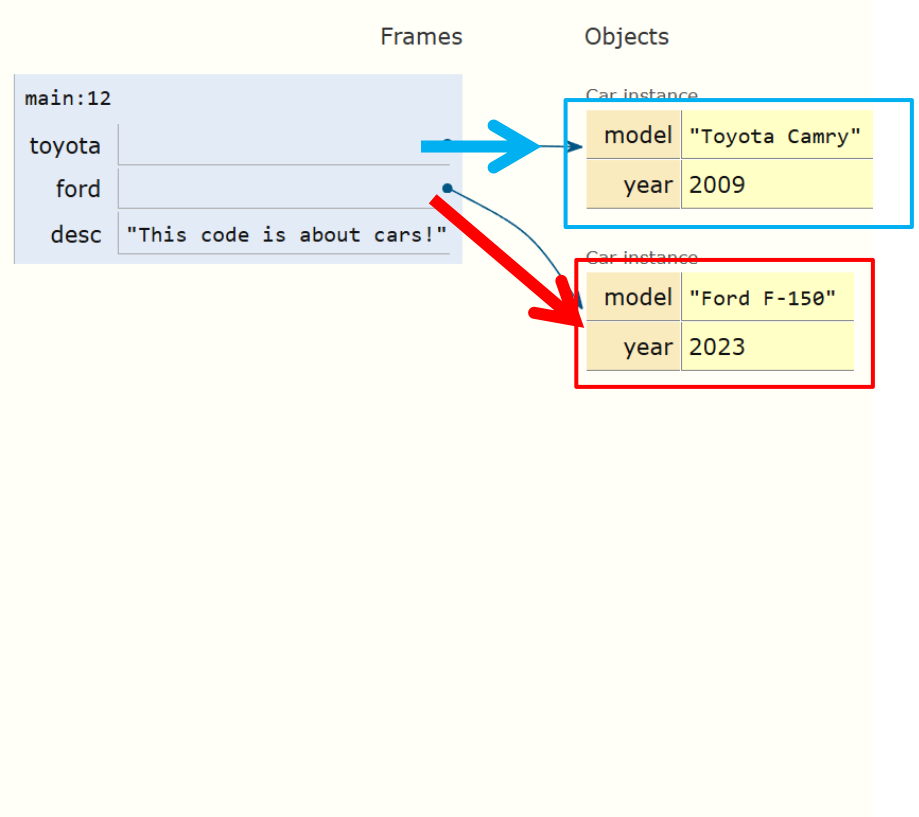


Objetos são tipos de dados por referência

- **Podem ter métodos**
  - **Sobre objetos**

```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+" anos.");
    System.out.println( "O Ford é de "+ford.getAge()+" anos.");
  }
}
```

```
class Car {
    String model;
    int year;
    int getYear()  { return this.year;  }

    Car(String m, int yr) {model=m; year=yr;}

    int getAge() {
      int currentYear = java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
      return getAge(currentYear);

    }
    int getAge(int currentYear) {
      return currentYear - year;
    }
}
```



Objetos da mesma classe “partilham” métodos e propriedades

Inicializar um objeto

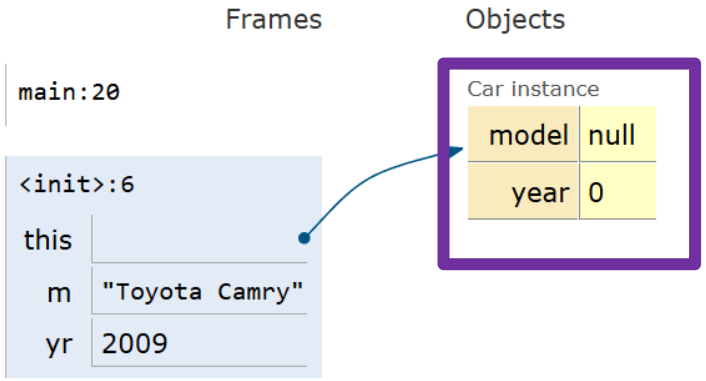
usando construtores das classes

Objetos são tipos de dados por referência

- Necessário "alocar" espaço (i.e., new)
- Usar construtores
  - Inicializar atributos

```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+" anos.");
    System.out.println( "O Ford é de "+ford.getAge()+" anos.");
  }
}
```

```
class Car {
    String model;
    int year;
    int getYear()  { return this.year;  }

    Car(String m, int yr) {model=m; year=yr;}

    int getAge() {
      int currentYear = java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
      return getAge(currentYear);

    }
    int getAge(int currentYear) {
      return currentYear - year;
    }
}
```

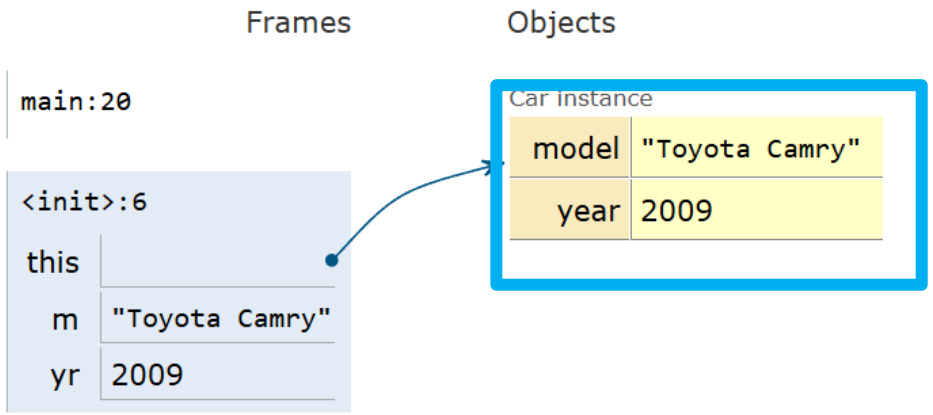


Objetos são tipos de dados por referência

- Necessário "alocar" espaço (i.e., new)
- Usar construtores
  - Inicializar atributos

```
public class CarTest {
  public static void main(String[] args) {
    Car toyota = new Car("Toyota Camry",2009);
    Car ford = new Car("Ford F-150",2023);
    String desc = "This code is about cars!";
    System.out.println( "O Toyota é de "+toyota.getAge()+" anos.");
    System.out.println( "O Ford é de "+ford.getAge()+" anos.");
  }
}
```

```
class Car {
    String model;
    int year;
    int getYear()  { return this.year;  }

    Car(String m, int yr) {model=m; year=yr;}

    int getAge() {
      int currentYear = java.util.Calendar.getInstance().get(java.util.Cal
      return getAge(currentYear);

    }
    int getAge(int currentYear) {
      return currentYear - year;
    }
}
```



Inicialização de membros

- Dentro de uma classe, a inicialização de variáveis pode ser feita na sua declaração.

```
class Measurement {
     int i = 25;
     char c = 'K’;
}
```

- Contudo, esta inicialização é igual para todos os objetos da classe
  - A solução mais comum é utilizar um construtor.

```
class Measurement {
     int i;
     char c;
     Measurement(int im, char ch) {
          i = im; c = ch;
     }
}
```

Construtor

- A inicialização de um objeto pode implicar a inicialização simultânea de diversos tipos de dados.
- Uma função membro especial, construtor, é invocada sempre que um objeto é criado.
- A instanciação é feita através do operador new.

```
Carro c1 = new Carro();
```

- O construtor é identificado pelo mesmo nome da classe.
- Este método pode ser *overloaded* (sobreposto) de modo a permitir diferentes formas de inicialização.

```
Carro c2 = new Carro("Ferrari", "430");
```

Construtor

- Não retorna qualquer valor
- Assume sempre o nome da classe
- Pode ter parâmetros de entrada
- É chamado apenas uma vez: na criação do objeto

```
public class Book {
   String title;
   int pubYear;

   public Book(String t, int py) {
        title = t;
        pubYear = py;
    }
    // ...
}
```

Questões?

- Qual o valor dos atributos de um objeto quando não foi definido nenhum construtor?

```
class Point
{
    public void display() {…}
    private double x, y;
};

Point p1 = new Point();
```

  - Quais os valores de x e y ?
  - É obrigatório iniciá-los no construtor?

Construtor por omissão

- Um construtor sem parâmetros é designado por *default constructor* ou construtor por omissão.
  - É automaticamente criado pelo compilador caso não seja especificado nenhum construtor.

```
class Book {
    String title;
    int pubYear;
}
```

```
Book m = new Book(); // ok
```

  - Se houver pelo menos um construtor associado a uma dada classe, o compilador já não cria o de omissão.

```
class Book {
    String title;
    int pubYear;
    Book(int py) { pubYear = py; }
}
```

```
Book m = new Book(); // ok?
```

https://tinyurl.com/4rbzxcd7

Construtores sobrepostos

- Permitem diferentes formas de iniciar um objeto de uma dada classe.

```
public class Book {
   public Book(String title, int pubYear)  {...}
   public Book(String title) {...}
   public Book() {...}
}

Book c1 = new Book("A jangada de pedra", 1986);
Book c2 = new Book("Galveias");
Book c3 = new Book();
```

Inicialização e Limpeza de Objetos

Alcance/Scope

- Uma variável pode ser utilizada desde o momento que é definida até ao final desse contexto
- Cada bloco pode ter os seus próprios objetos

```
{  int k = 10;
   {  int i = k+1;
   } // 'i' não é visível aqui
} // 'k' não é visível aqui
```

- Exemplo ilegal

```
{  int x = 12;
  {  int x = 96; /* erro! */
  }
}
```

Alcance de referências e objetos

- Exemplo com referências e objetos

```
{
        Book b1 = new Book("Memória de Elefante");
}
// 'b1' já não é visível aqui
```

- Neste caso a referência *b1* é libertada (removida) e o objeto deixa de poder ser usado
  - Será removido pelo **Garbage collector**

This

This é uma auto referencia do objecto

- This é uma referência do objecto actual
  - i.e. onde os métodos e constructores estão a ser executados
- Só pode ser usado em objectos
  - Não pode ser usado em métodos estáticos (static)
- Serve como referência para o próprio objecto
  - Passar como argumento
  - Retornar a própria referência (chaining)

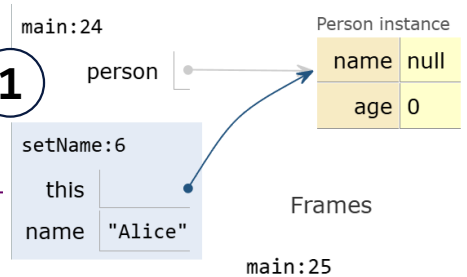
```
//Pass the current object as an argument
class Button {
  void setListener(Listener l) { /*...*/ }

  void init() {
    setListener(this);
  }
}


//Return the current object (method chaining)
class Builder {
  Builder add(String s) { /*...*/ return this; }
}
```

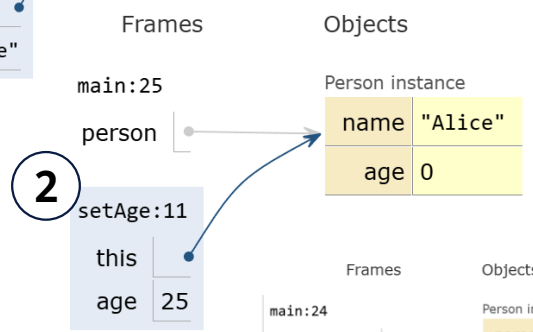
This é uma auto referencia do objecto

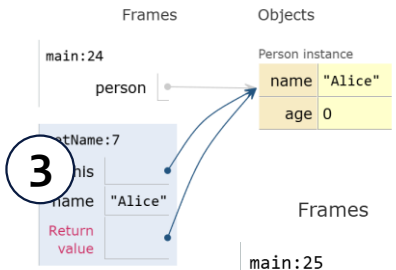
- Permite desambiguar entre parametros e propriedades de um objecto
- Invocar constructor na mesma class (mesmo nome , diferentes parametros)
- Para tornar explicito que estamos a usar o objecto actual

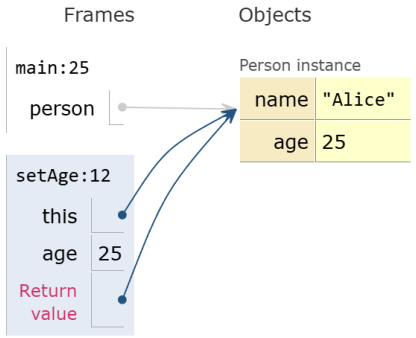
```
// Disambiguate fields from parameters/local variables
class Person {
  private String name;

  Person(String name) {
    this.name = name;   // field = parameter
  }
}

//Call another constructor in the same class
class Point {
  int x, y;

  Point() {
    this(0, 0);         // must be the first statement in the
constructor
  }

  Point(int x, int y) {
    this.x = x;
    this.y = y;
  }
}
```

```java
class Person {
    private String name;
    private int age;

    public Person setName(String name) {
        this.name = name;
        return this; // return current object
    }

    public Person setAge(int age) {
        this.age = age;
        return this; // return current object
    }

    public void introduce() {
        System.out.println("Hi, I'm " + name + " and I'm " + age + " years
old.");
    }
}

public class Main {
    public static void main(String[] args) {
        Person person = new Person();

        person.setName("Alice")
              .setAge(25)
              .introduce();
    }
}
```



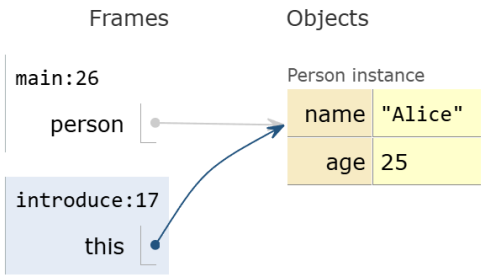






Print output (drag lower right corner to resize)

```
Hi, I'm Alice and I'm 25 years old.
```



[Link in pythonTutor (it also supports java)](#)

Invocar um construtor dentro de outro

- Quando escrevemos vários construtores podemos chamar um dentro de outro.
  - a referência this permite invocar sobre o mesmo objeto um outro construtor.

```
public Book(String title, int pubYear) {
        this.title = title;
        this.pubYear = pubYear;
}
public Book(String title) {
        this(title, 2000);
}
```

Duas formas:

- this.método()
- this(..)

- Esta forma só pode ser usada dentro de construtores;
  - neste caso this deve ser a primeira instrução a aparecer;
  - não é possível invocar mais do que um construtor this.

Chaining

- Chaining é a capacidade de encadear a invocação métodos, utilizando o estado/resultado anterior como entrada do próximo método.
- Outra utilização da referência this é para retornar, num dado método, a referência para esse objeto (permitindo este encadeamento).

https://www.geeksforgeeks.org/method-chaining-in-java-with-examples/

```
class A {

    private int a;
    private float b;

    A() { System.out.println("Calling The Constructor"); }

    public A setint(int a)
    {
        this.a = a;
        return this;
    }

    public A setfloat(float b)
    {
        this.b = b;
        return this;
    }

    void display()
    {
        System.out.println("Display=" + a + " " + b);
    }
}

// Driver code
public class Example {
    public static void main(String[] args)
    {
        // This is the "method chaining".
        new A().setint(10).setfloat(20).display();
    }
}
```

Chaining

- Chaining é a capacidade de encadear a invocação métodos, utilizando o estado/resultado anterior c
- O p re (p

htt
in-java-with-examples/

Method Chaining is the practice of calling different methods in a single line instead of calling other methods with the same object reference separately. Under this procedure, we have to write the object reference once and then call the methods by separating them with a (dot.).

```
obj.method1().method2().method3();
```

```
class A {

    private int a;
    private float b;

    A() { System.out.println("Calling The Constructor"); }

    public A setint(int a)
    {
        this.a = a;
        return this;
```

```
    public static void main(String[] args)
    {
        // This is the "method chaining".
        new A().setint(10).setfloat(20).display();
    }
}
```

Chaining

- Outra utilização da referência this é para retornar, num dado método, a referência para esse objeto.

```
class A {

    private int a;
    private float b;

    A() { System.out.println("Calling The Constructor"); }

    public A setint(int a)
    {
        this.a = a;
        return this;
    }

    public A setfloat(float b)
    {
        this.b = b;
        return this;
    }

    void display()
    {
        System.out.println("Display=" + a + " " + b);
    }
}

// Driver code
public class Example {
    public static void main(String[] args)
    {
        // This is the "method chaining".
        new A().setint(10).setfloat(20).display();
    }
}
```

https://www.geeksforgeeks.org/method-chaining-in-java-with-examples/

Chaining

- Outra utilização da referência this é para retornar, num dado método, a referência para esse objeto.

```
class A {

    private int a;
    private float b;

    A() { System.out.println("Calling The Constructor"); }

    public A setint(int a)
    {
        this.a = a;
        return this;
    }

    public A setfloat(float b)
    {
        this.b = b;
        return this;
    }

    void display()
    {
        System.out.println("Display=" + a + " " + b);
    }
}

// Driver code
```

```
// This is the "method chaining".
new A().setint(10).setfloat(20).display();
```

```
    }
}
```

https://www.geeksforgeeks.org/method-chaining-in-java-with-examples/

O conceito static

- Os métodos estáticos não têm associada a referência this.
- Assim, não é possível invocar métodos não estáticos a partir de métodos estáticos.
- É possível invocar um método estático sem que existam objetos dessa classe.
- Os métodos static têm a semântica das funções globais (não estão associadas a objetos).

Elementos estáticos

- As variáveis estáticas, ou variáveis de classe, são comuns a todos os objetos dessa classe.
- A sua declaração é precedida por static.
- A invocação é feita sobre o identificador da classe

```
class Test {
  public static int a=23;
  public static void someFunction() { … }
  // …
}

Test.someFunction();  // invocada sobre a classe
Test s1 = new Test();
Test s2 = new Test();
System.out.println(Test.a);
Test.a++; // s1.a e s2.a será 24
```

Inicialização de membros estáticos

- Se existir inicialização de membros estáticos esta toma prioridade sobre todas as outras.
- Um membro estático só é inicializado quando a classe é carregada (e só nessa altura)
  - quando for criado o primeiro objeto dessa classe ou quando for usada pela primeira vez.
- Podemos usar um bloco especial - inicializador estático - para agrupar as inicializações de membros estáticos

```
class Circulo {
  static private double lista[ ] = new double[100];
  static {  // inicializador estático
    // inicialização de lista[ ]
  }
}
```

Sobreposição (Overloading)

Mesmo nome, diferentes formas

Sobreposição (Overloading)

```
class Car {
    String model;
    int year;
    Car(String model, int year) {
        this.model = model;
        this.year = year;
    }

    public int getAge() {
        int currentYear = java.time.Year.now().getValue();
        return getAge( currentYear );
    }

    public int getAge( int currentYear) {
        return currentYear - year;
    }
}
```

Sobreposição (Overloading)

- Podemos usar o mesmo nome em várias funções
  - Desde que tenham argumentos distintos e que conceptualmente executem a mesma ação

```
void sort(int[] a);
void sort(Book[] b);
```

- A ligação estática verifica a assinatura da função (nome + argumentos)

Sobreposição (Overloading)

- Podemos usar o mesmo nome em várias funções
  - Desde que tenham argumentos distintos e que conceptualmente executem a mesma ação

```
void sort(int[] a);
void sort(Book[] b);
```

- A ligação estática verifica a assinatura da função (nome + argumentos)

- Não é possível distinguir funções pelo valor de retorno
  - porque é permitido invocar, p.e., void f() ou int f() na forma `f();`, em que o valor de retorno não é usado

Sobreposição (Overloading)

```
public class Test {
   public void someFunction(String[] s) {
      …
   }
   public int someFunction(String[] b) {
      …
   }
      // ...
   String[] someStrings = {“first string”, “another string”, “last”};
   someFunction(someStrings);
}
```

Problema ?

Problema com overloading

- Não é possível distinguir funções pelo valor de retorno
  - porque é permitido invocar, p.e., void f() ou int f() na forma f();, em que o valor de retorno não é usado

```
public class Test {
   public void someFunction(String[] s) {
                  …
   }
   public int someFunction(String[] b) {
                  …
   }
      // ...
   String[] someStrings = {“first string”, “another string”, “last”};
   someFunction(someStrings);
}
```

NÃO É POSSÍVEL

Visibilidade / Encapsulamento

Encapsulamento

- Ideias fundamentais da POO
  - Encapsulamento (Information Hiding)
  - Herança
  - Polimorfismo

- Encapsulamento
  - Separação entre aquilo que não pode mudar (interface) e o que pode mudar (implementação)
  - Controlo de visibilidade da interface (public, protected, *default*, private)

Encapsulamento

- Permite criar diferentes níveis de acesso aos dados e métodos de uma classe.
- Os níveis de controlo de acesso que podemos usar são, do maior para o menor acesso:
  - **public** - pode ser usado em qualquer classe
  - **protected** – visível dentro do mesmo package e classes derivadas
  - "omissão" – visível dentro do mesmo package
  - **private** – apenas visível dentro da classe

Modificadores/Selectores

- O encapsulamento permite esconder os dados internos de um objeto
  - Mas, por vezes é necessário aceder a estes dados diretamente (leitura e/ou escrita).

- Regras importantes!
  - Todos os atributos deverão ser privados.
  - O acesso à informação interna de um objeto (parte privada) deve ser efetuada sempre, através de funções da interface pública.

porquê?

Todos os atributos deverão ser privados.

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

```
public class Car {

  static int count = 0;

  private String model;
  private int year;
  private int matricula;

  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  public String getModel() {
    return model;
  }

  public int getMatricula() {
    return matricula;
  }

  public int getYear() {
    return year;
  }

  public int getAge() {
    int currentYear = java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  public int getAge(int currentYear) {
    return currentYear - year;
  }
```

Private ou protected

getters

Exemplo

- Um método de uma classe tem acesso a toda a informação e a todos os métodos dessa classe

```
class X {
  private int i;
  public void pub1( ) { /* . . . */ }
  private void priv1( ) { /* . . . */ }
  // ...
}

class XUser {
  private X myX = new X();
  public void teste() {
     myX.pub1(); // OK
     // myX.priv1();  Errado!
  }
}
```

Seletores/Modificadores (getters/setters)

- Seletor
  - Devolve o valor atual de um atributo

```
public float getRadius() { // ou public float radius()
    return radius;
}
```

- Modificador
  - Modifica o estado do objeto

```
public void setRadius(float newRadius) {
            // ou public void radius(float newRadius)
    this.radius = newRadius;
}
```

Métodos privados

- Internamente uma classe pode dispor de diversos métodos privados que só são utilizados internamente por outros métodos da classe.

```
// exemplo de funções auxiliares numa classe
class Screen {
    private int row();
    private int col();
    private int remainingSpace();
    // ...
};
```

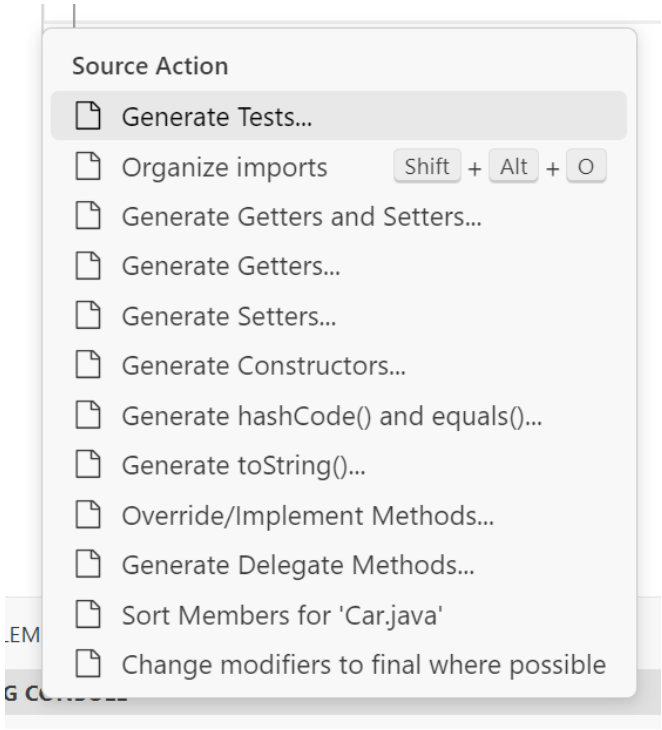
java.lang.Object
Métodos comuns a todos os objetos

Métodos comuns a todos os objetos

- Em Java, todos as classe derivam da super classe java.lang.Object
- Métodos desta classe:
  - toString()
  - equals()
  - hashcode()
  - getClass()
  - clone()
  - wait()
  - notify()
  - notifyAll()
  - ~~finalize~~()

Métodos comuns a todos os objetos

- Em Java, todos as classe derivam da super classe java.lang.Object
- Métodos desta classe:
  - toString()
  - equals()
  - hashcode()
  - getClass()
  - clone()
  - wait()
  - notify()
  - notifyAll()
  - ~~finalize~~()



toString()

- Todos os objetos em Java entendem a mensagem toString()

```
Book oneBook = new Book();
oneBook.setTitle("Turismo em Aveiro");
System.out.println(oneBook); // oneBook.toString()
```

```
Book@33909752
```

- Geralmente é necessário redefinir este método de modo a fornecer um resultado mais adequado.

```
@Override
public String toString() {
    return "Book: title=" + title + "; pubYear=" + pubYear;
}
```

```
Book: title=Turismo em Aveiro; pubYear=0
```

toString()

- Todos os objetos em Java entendem a mensagem toString()
- Geralmente é necessário redefinir este método de modo a fornecer um resultado mais adequado.

```
@Override
public String toString() {
  return "Car [model=" + model + ", year=" + year + ", matricula=" +
  matricula + ", getAge()=" + getAge() + "]";
}
```

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

toString()

```
Circulo c1 = new Circulo(1.5, 0, 0);
System.out.println( c1 );
```

```
Circulo@1afa3
```

c1.toString() é invocado automaticamente

- O método toString() deve ser sempre redefinido para ter um comportamento de acordo com o objeto

```
public class Circulo {
    // ....
    @Override
   public String toString() {
       return "Centro : ("  + centro.x() + ", "
               + centro.y() + ") " +      " Raio : " + raio;
    }
```

```
Centro : (1.5, 0) Raio : 0
```

equals()

- A expressão c1 == c2 verifica se as referências c1 e c2 apontam para a mesmo objeto
  - Caso c1 e c2 sejam variáveis automáticas a expressão anterior compara valores

- O método equals() testa se dois objetos são iguais

```
Circulo p1 = new Circulo(0, 0, 1);
Circulo p2 = new Circulo(0, 0, 1);
System.out.println(p1 == p2);         // false
System.out.println(p1.equals(p2));  // false (porquê?)
```

- equals() deve ser redefinido em concordância com os atributos dessa classe e é necessário para comparar objetos
  - Circulo, Ponto, Complexo ...

Problemas com equals()

- Propriedades da igualdade
  - reflexiva: x.equals(x) → true
  - simétrica: x.equals(y) ←→ y.equals(x)
  - transitiva: x.equals(y) AND y.equals(z) → x.equals(z)
- Devemos respeitar a assinatura **Object.equals(Object o)**

```
public class Circulo {
    //...
    @Override
    public boolean equals(Object obj) { //...
    }
}
```

- Problemas
  - E se 'obj' for null?
  - E se referenciar um objeto diferente de Circulo?

equals()

```
@Override
public boolean equals(Object obj) {
  if (this == obj)
    return true;
  if (obj == null)
    return false;
  if (getClass() != obj.getClass())
    return false;
  Car other = (Car) obj;
  if (model == null) {
    if (other.model != null)
      return false;
  }
  if (matricula != other.matricula)
    return false;
  return true;
}
```

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

Circulo.equals()

```
@Override
public boolean equals(Object obj) {
    if (this == obj)
        return true;
    if (obj == null)
        return false;
    if (getClass() != obj.getClass())
        return false;
    Circulo other = (Circulo) obj;
        // verify if the object’s attributes are equals
    if (centro == null) {
        if (other.centro != null)
            return false;
    } else if (!centro.equals(other.centro))
        return false;
    if (raio != other.raio)
        return false;
    return true;
}
```

hashCode()

- Sempre que o método **equals()** for reescrito, **hashCode()** também deve ser
  - Objetos iguais devem retornar códigos de hash iguais
- O objetivo do hash é ajudar a identificar qualquer objeto através de um número inteiro

```
// Circulo.hashCode() – Exemplo muito simples !!!
public int hashCode() {
    return raio * centro.x() * centro.y();
}
//..
Circulo c1 = new Circulo(10,15,27);
Circulo c2 = new Circulo(10,15,27);
Circulo c3 = new Circulo(10,15,28);
```

```
4050
4050
4200
```

  - **A construção de uma boa função de hash não é trivial. Para a sua construção recomendam-se outras fontes**

Funções de hash – conceitos base

- **Fundamental:** Objetos iguais têm de ter um valor de hash igual.

  (não funciona caso não se verifique)

hashCode()

```
@Override
public int hashCode() {
  final int prime = 31;
  int result = 1;
  result = prime * result + ((model == null) ? 0 : model.hashCode());
  result = prime * result + year;
  result = prime * result + matricula;
  return result;
}
```

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

Circulo.hashCode()

```
@Override
public int hashCode() {
     final int prime = 31;
     int result = prime
         + ((centro == null) ? 0 : centro.hashCode());
     long temp = Double.doubleToLongBits(raio);
     result = prime * result + (int) (temp ^ (temp >>> 32));
         // ^   Bitwise exclusive OR
         // >>> Unsigned right shift
     return result;
}
```

Classes Wrappers: necessárias para usar tipos primitivos como objetos (pex., nas Collections)

Tipos primitivos

- Não são objetos.
- Variáveis armazenam diretamente os dados.
- Dimensão em memória depende do tipo de dados.

```
int myNum = 5;                  // Integer (whole number)
float myFloatNum = 5.99f;       // Floating point number
char myLetter = 'D';            // Character
boolean myBool = true;          // Boolean
String myText = "Hello";        // String
```

https://www.w3schools.com/java/java_data_types.asp

Tipos primitivos e classes adaptadoras ("wrappers")

- As classes adaptadoras criam objetos para os tipos primitivos
- Contêm métodos especializados

Primitive Data Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean
char	Character

https://www.w3schools.com/java/java_wrapper_classes.asp

Boas práticas

Boas práticas: Classes

- Devemos usar uma nomenclatura do tipo Person, SomeClass, SomeLongNameForClass, …
- Java é uma linguagem case-sensitive (i.e. Exemplo != exemplo)
- Se classe for pública i.e., utilizável por outras classes
  - Deve ser declarada pública
  - O ficheiro Car.java deve conter uma classe pública denominada Car.

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

Boas práticas: Classes

- Devemos usar uma nomenclatura do tipo Person, SomeClass, SomeLongNameForClass, …
- Java é uma linguagem case-sensitive (i.e. Exemplo != exemplo)
- Se classe for **pública** i.e., utilizável por outras classes
  - Deve ser declarada pública
  - O ficheiro Car.java deve conter uma classe pública denominada Car.

```
public class Car {

  static int count = 0;

  String model;
  int year;
  int matricula;
  int getYear()  { return this.year;  }

  Car(String m, int yr) {model=m; year=yr;
      this.matricula = Car.count++;
  }

  int getAge() {
    int currentYear =
    java.util.Calendar.getInstance().get(java.util.Calendar.YEAR);
    return getAge(currentYear);

  }
  int getAge(int currentYear) {
    return currentYear - year;
  }
}
```

Boas práticas

- A semântica de construção de um objeto deve fazer sentido

```
Pessoa p = new Pessoa();   ☹
Pessoa p = new Pessoa("António Nunes"); 😐
Pessoa p = new Pessoa("António Nunes", 12244, dataNasc); ☺
```

- Devemos dar o mínimo de visibilidade pública no acesso a um objeto
  - Apenas a que for estritamente necessária
- Por vezes, faz mais sentido criar um novo objeto do que mudar os atributos existentes

```
Point p1 = new Point(2,3);
p1.set(4,5); ☹
```

Boas práticas

- Juntar membros do mesmo tipo
  - Não misturar métodos estáticos com métodos de instância
- Declarar as variáveis antes ou depois dos métodos
  - Não misturar métodos, construtores e variáveis
- Manter os construtores juntos, de preferência no início
- Se for necessário definir blocos static, definir apenas um no início ou no final da classe.

- A ordem dos membros não é importante, mas seguir convenções melhora a legibilidade do código

The End

Variáveis e tipos primitivos

Type	Size	Range	Default
boolean	1 bit	true or false	false
byte	8 bits	[-128, 127]	0
short	16 bits	[-32,768, 32,767]	0
char	16 bits	['\u0000', '\uffff'] or [0, 65535]	'\u0000'
int	32 bits	[-2,147,483,648 to 2,147,483,647]	0
long	64 bits	$[-2^{63}, 2^{63}-1]$	0
float	32 bits	32-bit IEEE 754 floating-point	0.0
double	64 bits	64-bit IEEE 754 floating-point	0.0